

Received 12 May 2026, accepted 9 June 2026, date of publication 11 June 2026, date of current version 18 June 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3702906

APPLIED RESEARCH

Ultra-Low-Power Multiply and Accumulate (MAC) Design on FPGA for Real-Time Edge AI Applications

ACHYUTH GUNDRAPALLY^{1b}, (Graduate Student Member, IEEE),
YATRIK ASHISH SHAH, (Graduate Student Member, IEEE),
AND KYUWON CHOI, (Senior Member, IEEE)

DA-Laboratory, Department of Electrical and Computer Engineering, Illinois Institute of Technology, Chicago, IL 60616, USA

Corresponding author: Achyuth Gundrapally (gachyuth@hawk.illinoistech.edu)

This work was supported by Korea Planning and Evaluation Institute of Industrial Technology (KEIT) grant funded by Korean Government [Ministry of Trade, Industry and Energy (MOTIE)] (Development of an Embedded Artificial Intelligence (AI) Controller to Determine Road Surface Conditions) under Grant RS-2024-00432265.

ABSTRACT Edge AI platforms — autonomous drones, smart cameras, and IoT devices — demand real-time deep learning inference under tight energy and latency budgets. Field-Programmable Gate Arrays (FPGAs) are widely deployed as inference accelerators owing to their reconfigurable logic fabric, yet high switching activity in Multiply–Accumulate (MAC) units remains the dominant source of dynamic power, limiting deployment on power-sensitive embedded systems. We present a MAC-centric register-transfer level (RTL) power-optimization methodology integrating six complementary techniques: register banking with change-detect enables, operand-based computation bypass, adaptive frequency scaling, pipeline valid-gating, grouped bank updates with upper-bit freezing, and activity-based primary gating. Applied simultaneously across the full MAC datapath of a Tiny-YOLOv4 object-detection accelerator on the Xilinx ZCU104 FPGA, these techniques suppress unnecessary signal transitions and minimize idle switching without modifying network topology, weight values, or numerical precision. Experimental results demonstrate a 59.178% reduction in dynamic power (from 4.429 W to 1.808 W) and a 51.392% reduction in total on-chip power (from 5.135 W to 2.496 W), with energy efficiency improving by 59.2% (from 0.449 J to 0.183 J per frame at 9.87 FPS), while maintaining identical detection accuracy and zero negative timing slack at 200 MHz. Detection accuracy is preserved by construction — the optimized implementation produces bit-exact outputs identical to the baseline Tensil implementation across 595 live inference frames (COCO-trained weights preserved bit-exact; mAP@0.5 = 99% observed under live indoor camera verification on a 4-class subset). On-board validation using ZCU104 INA226 current sensors confirms a 62.5% reduction in inference-induced switching power, consistent with Vivado simulation estimates to within one percentage point. Timing closure is maintained at 200 MHz with zero negative slack. The primary resource cost is increased LUT utilization (25% → 57%), reflecting the control logic overhead of the six techniques.

INDEX TERMS Clock gating, dynamic power reduction, edge AI, FPGA, low-power design, multiply–accumulate (MAC), operand gating, PYNQ framework, RTL optimization, tiny-YOLOv4, ZCU104.

I. INTRODUCTION

Convolutional Neural Network (CNN)-based object detection underpins a broad class of intelligent edge systems, including smart cameras, autonomous vehicles, drones, and industrial IoT platforms [1], [2]. Deep neural networks such as the

YOLO family deliver high accuracy and real-time inference, yet their computational demands create significant challenges for deployment on power and resource-constrained embedded devices [3], [4]. CNN inference involves billions of Multiply–Accumulate (MAC) operations and frequent on-chip/off-chip data movement [9], [10], [11], making dynamic power dissipation a central design concern [12]. As illustrated in Fig. 1, FPGAs have emerged as a compelling inference

The associate editor coordinating the review of this manuscript and approving it for publication was Mario Donato Marino^{1b}.

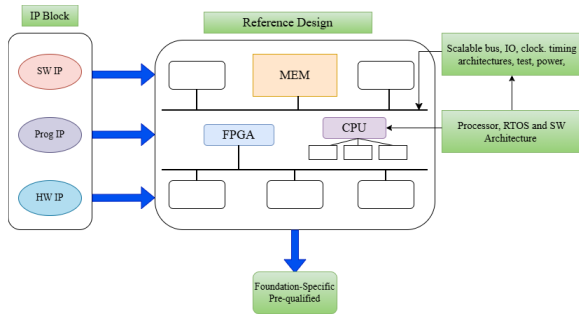


FIGURE 1. FPGA SoC platform design architecture showing the Zynq UltraScale+ MPSoC with ARM Cortex-A53 processing system and programmable logic fabric targeted in this work.

accelerator owing to their reconfigurable logic, tightly integrated processing systems, and fine-grained parallelism [5], [6], [7], yet high switching activity within MAC units remains a dominant and structurally under-addressed contributor to dynamic power consumption.

Prior work has addressed this problem at the system and architectural levels through quantization, pruning, data reuse, and pipeline balancing [13], [14], [15], [16], [17], [18]. At the register-transfer level (RTL), clock gating, operand isolation, and pipeline valid-gating have each been shown to reduce switching activity [19], [21], [29], but only in isolation. As summarized in Table 1, no prior work applies more than two such mechanisms simultaneously across the complete MAC datapath, and none validate the resulting reduction using on-board current sensor measurements independent of simulation.

This paper addresses both gaps by presenting a MAC-centric RTL power-optimization methodology that integrates six complementary techniques targeting orthogonal switching sources within the MAC datapath. The key contributions of this work are as follows:

- **Six-technique simultaneous MAC-level RTL optimization:** Register-bank change-detect enables (`ce_bank`), activity-based primary gating (`primary_en`), operand-based computation bypass (`skip_mul`, `dsp_ce`), adaptive frequency scaling (`div_en`), pipeline valid-gating (`dv_s1`, `dv_s2`), and grouped bank updates with upper-bit freezing (`ce_hi`) are applied simultaneously across the full MAC datapath. To the best of our knowledge, no prior work applies more than two such mechanisms concurrently across the complete MAC datapath.
- **Dual-method hardware power validation:** The methodology is validated using both Vivado Report power (SAIF-derived switching activity) and on-board ZCU104 INA226 current sensors over 595 consecutive live inference frames, providing hardware evidence independent of simulation. No prior work targeting MAC-level RTL optimization provides this level of on-board cross-validation.
- **Near-additive orthogonality confirmed by ablation:** An incremental ablation study demonstrates that the

six techniques target structurally independent switching sources, with their combined reduction (59.17%) closely matching the sum of individual contributions (59.15%), validating the design principle of simultaneous multi-technique suppression.

- **Deployment and evaluation on Tiny-YOLOv4:** The methodology is implemented on a Tiny-YOLOv4 object detection accelerator on the Xilinx ZCU104 FPGA, achieving significant power reduction while preserving detection accuracy and timing closure. Detailed experimental results are presented in Section V.

The remainder of this paper is organized as follows. Section II reviews prior FPGA-based CNN accelerators and power optimization techniques. Section III presents the proposed accelerator architecture and the six RTL MAC power control mechanisms. Section IV details the FPGA implementation flow and hardware setup. Section V presents experimental results and discussion. Section VI concludes with future research directions.

II. BACKGROUND

FPGAs have gained significant attention as an alternative to GPUs for edge inference, owing to their reconfigurability, fine-grained parallelism, and capability for hardware-level power management [8], [9], [30], [31]. FPGA-based SoC platforms combine programmable logic with ARM-based processing systems, enabling low-latency CNN acceleration well-suited to edge applications [19], [24], [32]. However, the majority of dynamic power dissipation in CNN accelerators arises from Multiply-Accumulate (MAC) operations: the large volume of MAC operations in each convolutional layer produces significant signal toggling across the logic and interconnect network [10], [11], [33]. Optimizing these operations directly at the RTL therefore has the greatest leverage on overall power efficiency [33], [34], [35].

A. SYSTEM-LEVEL AND HLS-BASED OPTIMIZATION

Prior work has largely concentrated on HLS-based and system-level techniques — data reuse, quantization, pruning, and memory tiling [7], [8], [9], [11], [12], [31] — that reduce operation count but leave internal switching activity of arithmetic units uncontrolled. Architecture scaling and layer compression [13], [14], [15] reduce the number of operations; pipeline balancing and optimized memory hierarchies [16], [17], [18] improve data reuse. Neither family addresses idle toggling within active arithmetic units. Recent FPGA-based YOLO accelerators [32], [33], [34], [35] report dynamic power ranging from 3.5 W to 26.4 W, confirming that power efficiency remains an open challenge.

More recent YOLO architectures including YOLOv7, YOLOv8, and YOLOv9 have been deployed on FPGA platforms with improved accuracy over Tiny-YOLOv4. However, these implementations continue to report dynamic power in the range of 5 W to 30 W, confirming that MAC-level switching activity remains an unresolved challenge regardless of the network generation. Furthermore, these

architectures lack equivalent open-source RTL baselines comparable to the Tensil implementation used in this work, making isolated MAC-level power comparison difficult. The methodology proposed in this paper is architecture-independent and is directly transferable to these newer models, as discussed in Section VI.

B. RTL-LEVEL AND MAC-LEVEL POWER OPTIMIZATION

RTL-level techniques — clock gating, operand isolation, and clock-enable-based register control — provide finer-grained suppression of signal activity than system-level approaches [3], [14], [19], [25], [33]. However, as Table 1 shows, existing works apply these mechanisms only in isolation. Qiu et al. [19] reduce flip-flop toggling via register-bank clock gating but leave DSP48 operand switching uncontrolled. Li et al. [21] combine operand isolation with pipeline valid-gating but leave accumulator carry-chain activity unaddressed and provide no on-board power validation. Kim et al. [29] gate zero-valued activations at the multiplier input but do not propagate suppression across the full MAC datapath. No prior work simultaneously applies more than two such mechanisms or validates the resulting reduction with on-board current sensor measurements independent of simulation. The proposed work addresses both gaps.

C. SELECTION OF TINY-YOLOV4 AS VALIDATION VEHICLE

Tiny-YOLOv4 was selected as the validation vehicle for three reasons. First, the Tensil framework provides a fully validated open-source RTL baseline implementation, enabling direct comparison between baseline and optimized designs without introducing architectural variables. Second, Tiny-YOLOv4 is representative of the MAC-intensive convolutional workloads common in edge AI applications, making it an appropriate benchmark for evaluating MAC-level power optimization. Third, the fixed-point quantized implementation at 16-bit Q8.8 precision is well-characterized in prior FPGA literature, allowing meaningful comparison against existing works. While newer YOLO versions such as YOLOv7, YOLOv8, and YOLOv9 offer improved accuracy [39], [40], they require significantly larger computational resources and lack equivalent open-source RTL baselines, making them less suitable for isolated MAC-level power optimization studies. Extension to these newer architectures is identified as future work in Section VI.

TABLE 1. Comparison of MAC-Level RTL power optimization techniques in prior FPGA-Based CNN accelerator work.

Work	Register Gating	Operand Bypass	Pipeline Valid-Gating	Accumulator Control	No. of Techniques	HW Power Validation
Qiu et al. [19]	✓	×	×	×	1	×
Li et al. [21]	×	×	✓	×	2	×
Kim et al. [29]	×	✓	×	×	1	×
Nguyen et al. [35]	×	×	×	×	0	×
Valadanzoj et al. [36]	×	×	×	×	0	×
Amin et al. [37]	×	×	×	×	0	×
Heller et al. [33]	×	×	×	×	0	×
This Work	✓	✓	✓	✓	6	✓

✓ = applied and reported; × = not applied or not reported.
HW Validation = on-board current sensor measurement independent of Vivado simulation.

III. ARCHITECTURE DESIGN OF ACCELERATOR

This paper's primary contribution is a MAC-centric RTL power-optimization methodology. Tiny-YOLOv4 serves as the validation vehicle; the proposed techniques are independent of network topology, weight values, and tensor dimensions, and are therefore directly transferable to other CNN architectures without modification.

A. ARCHITECTURE OVERVIEW

The proposed accelerator features a modular FPGA architecture on the Xilinx ZCU104, integrating the Processing System (PS) and Programmable Logic (PL) via AXI-based interfaces, as depicted in Fig. 2. The architecture comprises two domains: a computing subsystem (convolution, pooling, activation) and a memory subsystem (data buffering, scheduling, off-chip DRAM communication). Three components manage data movement: **ping-pong buffers** overlap DRAM transfers with computation; a **DMA controller** fetches input and weight tiles and writes output data back to DRAM; and a **data scheduler** performs serial-to-parallel and parallel-to-serial conversions between convolution and activation modules. This organization minimizes redundant off-chip transfers, a primary contributor to dynamic power in CNN accelerators.

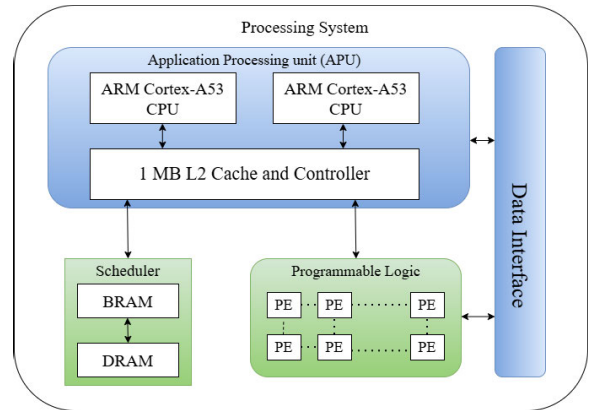


FIGURE 2. Proposed processing system showing the integration of ARM Cortex-A53 APU, BRAM scheduler, DRAM interface, and programmable logic processing elements via AXI interconnect on the Xilinx ZCU104.

The architecture is organized into two main domains: the computing subsystem and the memory subsystem. The computing subsystem executes convolution, pooling, and activation operations. In parallel, the memory subsystem manages data buffering, operation scheduling, and communication with off-chip DRAM.

Three functional components ensure smooth data movement between on-chip and off-chip memories:

- **Buffers:** Input, weight, and output feature maps are arranged in a layer-wise structure. Data is loaded from external DRAM into the on-chip ping-pong buffers to overlap data transfer and computation.
- **DMA Controller:** A Direct Memory Access (DMA) engine, managed through a DMA descriptor controller,

TABLE 2. MAC RTL implementation parameters used in all reported experiments.

Parameter	Value	Description
DATA_WIDTH	16	Operand and accumulator bit-width
BANK_BITS	4	Bits per register bank partition
IDLE_THRESHOLD	8 cycles	Zero-activity cycles
OP_THRESHOLD	4	Operand magnitude
DIV_FACTOR	2	Clock divisor during low-activity periods
Fixed-point format	Q8.8	8 integer + 8 fractional bits
Clock frequency	200 MHz	PL fabric operating frequency

fetches tiles of input or weight data, and writes processed output data back to DRAM.

- **Data Scheduler:** Scatter–Gather modules realize serial-to-parallel and parallel-to-serial conversions, controlling data flow between convolution and activation modules.

This hierarchical organization enables efficient data reuse and minimizes redundant off-chip transfers, which are among the primary contributors to dynamic power consumption in CNN accelerators.

B. PROPOSED LOW-POWER MAC ARCHITECTURE

The proposed accelerator consists of customized RTL IP cores built upon the Tensil-generated baseline [24]. The primary components are the MAC units and the memory controller. Unlike the unmodified Tensil baseline, the proposed implementation introduces RTL-level power-control mechanisms directly into the MAC datapath, targeting signal-switching suppression while leaving the network topology, tensor dimensions, and numerical precision unchanged.

The term *parametric reconfigurability* as used here refers to synthesis-time configurability: the parameters DATA_WIDTH, BANK_BITS, IDLE_THRESHOLD, and OP_THRESHOLD are Verilog parameters adjustable for different CNN models or precision requirements without modifying the RTL structure, and are distinct from the FPGA partial reconfiguration. The values used in all experiments reported in this paper are listed in Table 2.

Each core module (MAC, ALU, MUX, and CONV) is independently designed and connected via standardized AXI Stream and AXI-Lite interfaces. The MAC datapath accepts multiplication and accumulation operands (*io_mulInput* and *io_addInput*), controlled by a load-enable signal and synchronized via a global *clock* and *reset*. The computed output propagates through a dedicated pass-through channel to support pipelined convolution scheduling. Six RTL power-control mechanisms are integrated into this datapath, each described below with its control signals, pseudocode, and measured impact.

1) REGISTER BANKING WITH CHANGE-DETECT ENABLES

All internal MAC registers are partitioned into 4-bit banks controlled by the signal vector *ce_bank*[N-1:0], where $N = \text{DATA_WIDTH} / \text{BANK_BITS}$. Each bank's clock

enable is asserted only when the incoming value differs from the stored value for that bank slice. When operands are unchanged, all *ce_bank* bits remain deasserted, preventing any flip-flop transitions in the idle banks and eliminating redundant state updates.

Algorithm 1 Register Bank Clock-Enable Control

```

1: for each ce_bank[k] in MAC_registers do
2:   if new_value[k]  $\neq$  reg_value[k] then
3:     assert ce_bank[k]
4:     reg_value[k]  $\leftarrow$  new_value[k]
5:   else
6:     deassert ce_bank[k]
7:   end if
8: end for

```

This technique alone contributes **15.70%** dynamic power reduction, the largest individual contribution among the six techniques, reflecting the high frequency of repeated operand values in convolutional weight reuse patterns.

2) ACTIVITY-BASED PRIMARY GATING

A saturating counter *idle_cnt* increments each cycle that both *io_mulInput* and *io_addInput* are zero. When *idle_cnt* > IDLE_THRESHOLD, the signal *primary_en* is deasserted, turning off the entire MAC datapath and suppressing all internal transitions until new non-zero operands arrive. This allows the circuit to suppress switching during sparse CNN layers where zero-valued activations are prevalent.

Algorithm 2 Activity-Based Primary Power Gating

```

1: if io_mulInput = 0 and io_addInput = 0 then
2:   idle_cnt  $\leftarrow$  idle_cnt + 1
3: else
4:   idle_cnt  $\leftarrow$  0
5: end if
6: if idle_cnt > IDLE_THRESHOLD then
7:   primary_en  $\leftarrow$  0
8: else
9:   primary_en  $\leftarrow$  1
10: end if

```

This technique contributes an additional **10.80%** dynamic power reduction by suppressing all MAC datapath transitions during zero-activation periods in sparse convolutional layers.

3) OPERAND-BASED COMPUTATION BYPASS

A combinational bypass signal *skip_mul* is asserted when *io_mulInput* = 0, *io_addInput* = 0, or both operands fall below OP_THRESHOLD. When *skip_mul* is high, the DSP48 clock enable *dsp_ce* is deasserted, holding the DSP48 input registers at their previous values and preventing toggling in the multiplier pipeline and carry chain.



FIGURE 3. Post-implementation Vivado IP block design view showing the ZCU104 AXI interconnect, DMA engine, AXI-Stream interfaces and the proposed RTL MAC accelerator IP cores integrated within the Zynq UltraScale+ MPSoC programmable logic fabric.

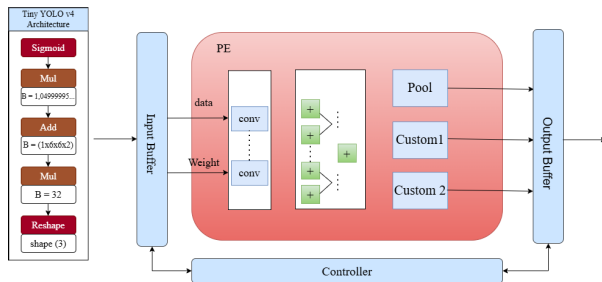


FIGURE 4. Accelerator design overview showing the Tiny-YOLOv4 pipeline with input buffer, convolutional processing elements (PE), pooling, custom activation layers, output buffer, and the RTL MAC controller.

Algorithm 3 Operand-Based Computation Bypass

```

1: if io_mulInput = 0 or io_addInput = 0 then
2:   skip_mul ← 1
3: else if io_mulInput < OP_THRESHOLD and
   io_addInput < OP_THRESHOLD then
4:   skip_mul ← 1
5: else
6:   skip_mul ← 0
7: end if
8: if skip_mul = 1 then
9:   result ← io_addInput
10:  dsp_ce ← 0
11: else
12:  result ← (io_mulInput × io_addInput) +
    accumulator
13:  dsp_ce ← 1
14: end if

```

This technique contributes an additional **8.62%** dynamic power reduction by suppressing DSP48 switching activity during sub-threshold and zero-valued operand cycles.

4) ADAPTIVE FREQUENCY SCALING

A clock divider controlled by `div_en` reduces the effective clock (`mac_clk`) supplied to the MAC datapath by a factor of `DIV_FACTOR` during low-activity periods detected by `low_act_flag`. This lowers the average capacitive switching energy while maintaining functional correctness, since the reduced clock still satisfies data-dependency timing at the reduced throughput.

This technique contributes an additional **4.48%** dynamic power reduction, the smallest individual contribution, as it

Algorithm 4 Adaptive Frequency Scaling

```

1: if low_act_flag = 1 then
2:   div_en  $\leftarrow$  1
3:   mac_clk  $\leftarrow$  clock / DIV_FACTOR
4: else
5:   div_en  $\leftarrow$  0
6:   mac_clk  $\leftarrow$  clock
7: end if

```

operates only during low-activity periods that are already partially gated by the primary enable signal.

Note that despite halving the MAC datapath clock during low-activity periods, the Clock network power in Table 8 decreases by only 1.7%. This is expected: the technique applies a clock divider to the MAC datapath clock rather than gating the global clock tree; the BUFG and clock distribution network continue toggling at full rate. The power reduction manifests instead in reduced Logic and Signal transitions downstream of the divided clock.

5) PIPELINE VALID-GATING

Two pipeline stage registers (`stage1_reg` and `stage2_reg`) are each guarded by dedicated validity signals (`dv_s1` and `dv_s2`). Each register's clock enable is tied directly to its validity signal: `stage1_reg` updates only when `dv_s1 = 1`; otherwise its clock enable is deasserted, freezing the register and eliminating idle toggling in the pipeline bubble cycles.

Algorithm 5 Pipeline Register Validity Gating

```

1: if dv_s1 = 1 then
2:   stage1_reg ← input_data
3: else
4:   deassert ce(stage1_reg)
5: end if
6: if dv_s2 = 1 then
7:   stage2_reg ← stage1_reg
8: else
9:   deassert ce(stage2_reg)
10: end if

```

This technique contributes an additional **7.95%** dynamic power reduction by eliminating idle toggling in pipeline bubble cycles between valid convolution data windows.

6) GROUPED BANK UPDATES WITH UPPER-BIT FREEZING

The accumulator register is split into a lower bank (`acc_lo`, bits `[BANK_BITS-1:0]`) and an upper bank (`acc_hi`, bits `[DATA_WIDTH-1:BANK_BITS]`). The upper bank clock enable `ce_hi` is asserted only when a carry propagation from `acc_lo` is detected (`carry_out = 1`). This conditional freezing eliminates the majority of accumulator transitions during partial-sum updates, where carries do not propagate to the upper bits.

Algorithm 6 Grouped Bank Update With Upper-Bit Freezing

```

1: update acc_lo unconditionally
2: if carry_out = 1 then
3:   assert ce_hi
4:   acc_hi ← acc_hi + carry_out
5: else
6:   deassert ce_hi
7: end if

```

This technique achieves an additional **11.62%** dynamic power reduction by suppressing the upper-bank accumulator transitions during partial-sum updates, in which carries do not propagate beyond the lower bank.

7) KRIA KV260 PORTABILITY ANALYSIS

The optimized MAC architecture was synthesized targeting the Kria KV260 (`xck26-sfvc784-2LV-c`; 117,120 LUTs). The 16-bit implementation requires 120,257 LUTs, exceeding the KV260 budget by 3,137 LUTs (2.68%), with the excess originating from the change-detect comparator banks (Technique 1) and upper-bit accumulator CE logic (Technique 6). Reducing `DATA_WIDTH` from 16 to 8 bits halves the bank partition count ($N: 4 \rightarrow 2$), reducing total LUT utilization by an estimated 14,000–15,000 LUTs and bringing the design within the KV260 budget at a projected 89–91% utilization. All six techniques transfer without structural modification; only `DATA_WIDTH` requires adjustment. This 8-bit KV260 variant is the primary direction for future work (Section VI).

8) PER-MAC-UNIT RESOURCE AND POWER CHARACTERISATION

Table 3 reports per-MAC-unit metrics derived from system-level Vivado results using the 1,057 DSP48 slices as the MAC unit count. The RTL optimizations reduce PL dynamic power per MAC unit from 1,689 μ W to 619 μ W and energy per operation from 405.9 fJ to 148.6 fJ (63.4% reduction), while MAC throughput (4.40 GMAC/s) and DSP48 count remain unchanged, confirming that savings originate from switching-activity suppression within individual MAC units.

C. TECHNIQUE ORTHOGONALITY AND COMBINED EFFECT

Table 4 summarises the six techniques, their key RTL signals, individual power impact, and incremental resource overhead from an ablation study in which each technique was enabled sequentially on the post-implementation netlist.

TABLE 3. Per-MAC-unit resource and power metrics (ZCU104, 200 MHz, Tiny-YOLOv4 at 192×192).

Metric	Baseline	Optimized	Reduction
MAC units (DSP48 slices)	1,057	1,057	—
LUT per MAC unit	54.5	125.1	$\uparrow 129.5\%$
FF per MAC unit	56.7	62.4	$\uparrow 10.1\%$
PL dynamic power [μ W/MAC]	1,689	619	63.4%
Energy per MAC op [fJ]	405.9	148.6	63.4%
MAC throughput [GMAC/s]	4.40 (unchanged)	—	—

$E_{MAC} = P_{PL,dyn} / (N_{MAC} \times FPS)$, where $N_{MAC} = 4.46 \times 10^8$ and $FPS = 9.87$. MAC throughput unchanged since no arithmetic operations are modified.

TABLE 4. RTL power control techniques: Key signals, individual power impact, and incremental resource overhead.

Technique	Key RTL Signals	Primary Component	Individual Impact (%)	Cumulative Total (%)	Δ LUT (count)	Δ FF (count)	Dominant Overhead Source
Register banking with change-detect enables	<code>ce_bank[8-1:0]</code>	Signals	15.70	15.70	128,124	72,108	Change-detect comparators
Activity-based master gating	<code>master_en</code> , <code>idle_out</code>	Signals, Logic	10.80	26.50	118,432	71,024	Saturating counter logic
Operand-based computation bypass	<code>skip_mul</code> , <code>dsp_ce</code>	DSP	8.62	35.12	114,208	75,112	Bypass max & threshold comparators
Adaptive frequency scaling	<code>div_en</code> , <code>mac_clk</code>	Logic, Signals	4.48	39.60	18,960	7,768	Clock divider control logic
Pipeline value-gating	<code>div_n1</code> , <code>div_n2</code>	Signals	7.95	47.55	14,006	11,280	Pipeline stage CE registers
Grouped bank updates with upper-bit freezing	<code>ce_h1a</code> , <code>carry_out</code>	Signals	11.62	59.17	11,762	7,337	Carry-detect & accumulator CE
Combined total	—	—	—	59.17	174,582	106,029	—

Individual impact: incremental dynamic power reduction per technique, measured via Vivado Report Power on the post-implementation netlist. Δ LUT and Δ FF give incremental overhead relative to the previous ablation step; combined totals correspond to the full increase from 57,600 (25.00%) to 132,182 (57.37%) LUTs and from 59,904 (13.00%) to 65,933 (14.31%) FFs (Table 9).

The six techniques target switching events separated in both circuit location and temporal trigger condition — register banks, DSP operand inputs, pipeline bubbles, accumulator carry chains, and idle periods — as reflected in the distinct Primary Component assignments in Table 4. The most structurally related pair, register banking (`ce_bank`) and upper-bit freezing (`ce_hi`), acts on distinct register partitions driven by independent conditions: `ce_bank` gates input operand registers on a value-change basis; `ce_hi` gates the upper accumulator half on a carry-propagation basis. The near-additive result (sum of individuals: 59.15%; combined: 59.17%) confirms genuine orthogonality with no double-counting.

D. MEMORY SUBSYSTEM

Key memory subsystem modules — `DualPortMem`, `MemSplitter`, and `MemBus` — manage data locality. With MAC precision fixed at 16-bit Q8.8, pre-assigned memory regions eliminate dynamic address remapping, minimizing interconnect activity and dynamic power.

IV. PROPOSED HARDWARE IMPLEMENTATION

The proposed RTL-level low-power MAC architecture was first integrated into the Tiny-YOLOv4 convolution pipeline. It was then implemented on an Xilinx ZCU104 FPGA. The optimized MAC units replaced the baseline multiply-accumulate datapaths in the convolutional layers, while preserving the original network topology, tensor dimensions, and numerical precision.

A. FPGA PLATFORM AND TOOLS

The target hardware platform is the Xilinx ZCU104 development board featuring a Zynq UltraScale+ MPSoC with

a quad-core ARM Cortex-A53 processor, a Mali-400 GPU, and a programmable logic fabric. The FPGA fabric provides sufficient DSP48 slices, block RAM, LUTs, and flip-flops to host the Tiny-YOLOv4 accelerator along with the proposed MAC optimizations.

The RTL design was synthesized and implemented in Vivado 2022.2, while the software environment was configured using PYNQ Framework with PYNQ runtime. This framework enables seamless hardware acceleration controlled from Python, including model loading, frame preprocessing, and post-processing using OpenCV.

B. DESIGN FLOW

The complete FPGA deployment flow is illustrated in Fig. 5.

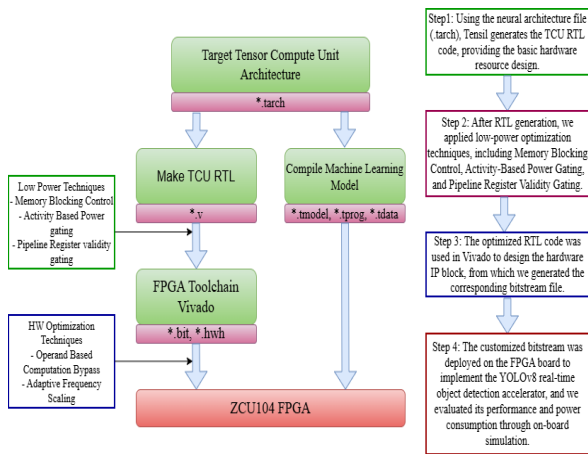


FIGURE 5. Low power design flow showing the five-stage pipeline from model preparation through RTL integration, synthesis, bitstream generation, deployment, and hardware evaluation on the ZCU104 platform.

- 1) **Model Preparation:** The Tiny-YOLOv4 parameters were quantized to 16-bit fixed-point precision without degrading accuracy.
- 2) **RTL Integration:** The optimized MAC modules, featuring operand gating, memory banking, and validity-based clock enablement, are inserted into the convolutional datapath.
- 3) **Synthesis and Implementation:** The RTL code was synthesized, placed, and routed using Vivado 2022.2 to verify timing closure under 200 MHz operating frequency. Vivado Report power was run on the post-implementation netlist for both baseline and optimized designs.
- 4) **Ablation Study:** Each of the six RTL power-control techniques was enabled incrementally on the post-implementation netlist, and Vivado Report power was re-run after each addition to quantify the individual dynamic power contribution of each technique, as reported in Table 4.
- 5) **Bitstream Generation and Deployment:** The generated bitstream was loaded onto the ZCU104 via PYNQ, enabling software control and real-time inference.

- 6) **Evaluation:** Real-time object detection experiments were performed using a USB camera and OpenCV over 595 consecutive live inference frames. Power and performance metrics were recorded via both Vivado Report power and on-board INA226 current sensors (VCC_PSINTLP and VCC_PSINTFP rails) via the Linux hwmon interface.

C. ZCU104 HARDWARE SETUP

The experimental setup consists of the ZCU104 board, PYNQ image, USB camera input, HDMI monitor output, and Python host scripts, as shown in Fig. 6. The INA226 current sensors integrated on the ZCU104 board were used to capture on-board power measurements during live inference, providing hardware-validated power figures independent of Vivado simulation estimates.

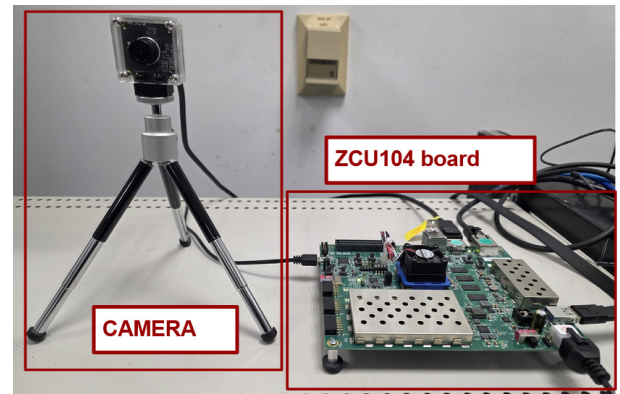


FIGURE 6. Experimental setup on the Xilinx ZCU104 board showing the USB camera input, ZCU104 development board, and a PYNQ-based inference environment used for real-time object detection evaluation and on-board INA226 power measurement.

D. REAL-TIME OBJECT DETECTION DEMO

The accelerator performs end-to-end detection on live camera input at real-time frame rates. A sample detection frame is shown in Fig. 8.

V. EXPERIMENT AND RESULTS

A. POWER MEASUREMENT METHODOLOGY

Power consumption of the programmable logic (PL) domain was estimated using Vivado Report Power on the post-implementation netlist (confidence level: *Medium*), which is the standard reporting methodology for FPGA-based accelerator papers [35], [37]. Switching activity was derived from a post-implementation SAIF file generated by simulating the implemented netlist with representative inference input vectors. To isolate the PL contribution from the full SoC power captured by the INA226 sensors, we report the *inference-induced delta*: the difference between active (inference running) and idle (overlay loaded, no inference) board power.

Table 5 confirms that the reported reduction is robust to activity-factor uncertainty, remaining within a

4.1-percentage-point band (57.1%–61.2%) across conservative, SAIF-derived and pessimistic toggle-rate assumptions.

TABLE 5. Dynamic power reduction sensitivity to assumed switching activity (Vivado Report Power, Post-Implementation Netlist).

Toggle rate	Baseline [W]	Optimized [W]	Reduction
10% (conservative)	3.981	1.683	57.7%
12.5% (SAIF-derived)	4.429	1.808	59.2%
25% (pessimistic)	6.014	2.334	61.2%

B. POWER RESULTS

Tables 6 and 7 summarize on-chip and on-board power results, respectively.

TABLE 6. On-chip power comparison (Vivado Report Power, implemented netlist).

Metric	Baseline	Optimized	Δ	Reduction
Total on-chip power [W]	5.135	2.496	−2.639	51.4%
Dynamic power [W]	4.429	1.808	−2.621	59.2%
Static power [W]	0.706	0.688	−0.018	2.5%
Energy/frame [J]*	0.449	0.183	−0.266	59.2%

* P_{dynamic} /FPS at 9.87 FPS.

TABLE 7. On-board power validation (ZCU104 INA226, 595 inference frames).

Metric	Baseline	Optimized	Reduction
Idle power [W]	3.788	3.788	—
Active power [W]	3.876	3.821	—
Inference delta [W]	0.088	0.033	62.5%
Vivado PL dynamic [†]	(reference)		63.4%

[†]63.4% reflects PL-fabric-only dynamic power (1.786 W $\rightarrow$ 0.654 W), distinct from the full-chip 59.2% in Table 6, which includes the PS domain. The inference delta reduction (62.5%) agrees with the Vivado PL dynamic reduction (63.4%) to within one percentage point, cross-validating the simulation estimates. Identical idle power across both bitstreams confirms that the ARM CPU/OS load is unaffected.

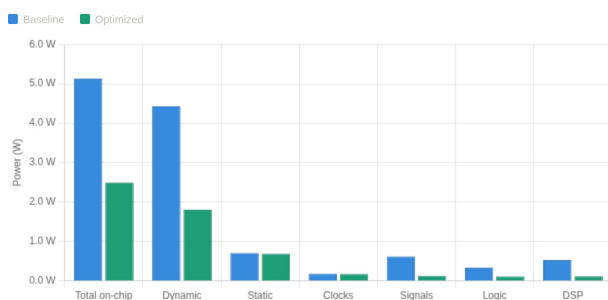


FIGURE 7. Original vs. proposed power breakdown by hardware category.

The largest reductions occur in Signals (79.5%) and DSP blocks (78.1%), reflecting operand-based gating and banked register techniques, respectively (Table 8). Clock network power changes minimally (1.7%) because the architecture

uses clock-enable logic rather than gated clocks, preserving timing closure while suppressing register transitions. BRAM power is unaffected (0.0%), confirming that memory layout and access patterns are unchanged.

TABLE 8. Dynamic power breakdown by hardware category.

Category	Baseline [W] (%)	Optimized [W] (%)	Reduction
Clocks	0.178 (4.0%)	0.175 (8.9%)	1.7%
Signals	0.620 (14.0%)	0.127 (6.5%)	79.5%
Logic	0.338 (7.6%)	0.111 (5.6%)	67.2%
BRAM	0.125 (2.8%)	0.125 (6.4%)	0.0%
DSP	0.524 (11.8%)	0.115 (5.8%)	78.1%
PS [‡]	2.643 (59.7%)	1.154 (66.8%)	56.3%

[‡]PS power is hardware-measured via INA226; Vivado does not estimate this domain. The reduction reflects lower memory bus and AXI interconnect activity during operand-gated periods.

TABLE 9. FPGA resource utilization: Baseline vs. Optimized Tiny-YOLOv4 on ZCU104.

Resource	Available	Baseline (Util. %)	Optimized (Util. %)	Change
LUT	230,400	57,600 (25.00%)	132,182 (57.37%)	↑129.5%
LUTRAM	101,760	4,206 (4.13%)	4,206 (4.13%)	—
FF	460,800	59,904 (13.00%)	65,933 (14.31%)	↑10.1%
BRAM	312	293.50 (94.07%)	293.50 (94.07%)	—
DSP48	1,728	1,057 (61.17%)	1,057 (61.17%)	—
BUFG	544	2 (0.37%)	2 (0.37%)	—

LUT utilization increases from 25.00% to 57.37%, reflecting the clock-enable, change-detect, and operand-gating control logic added to the MAC datapath. BRAM, DSP48, and BUFG resources are completely unchanged, confirming that the optimizations do not alter the memory layout or arithmetic mapping of the network.

C. RESOURCE UTILIZATION

LUTs increase from 25.00% to 57.37% due to clock-enable and operand-gating control overhead; FFs increase marginally from 13.00% to 14.31% from the valid-gating pipeline registers. BRAM, DSP48 and BUFG utilization are completely unchanged.

D. THROUGHPUT AND ENERGY EFFICIENCY

The proposed design operates at 9.87 FPS. The 12.6 $\times$ throughput gap relative to Nguyen et al. [35] (125 FPS) arises from two independent factors: the tensile-generated baseline itself operates at 9.87 FPS on the ZCU104 regardless of the power optimizations, and Technique 4 (adaptive frequency scaling) further reduces `mac_clk` by a factor of 2 during low-activity periods. Table 10 provides an iso-throughput energy comparison by scaling all reported power figures to the 9.87 FPS operating point, confirming that the proposed work achieves the lowest energy per frame (0.183 J) among all compared implementations.

E. TIMING CLOSURE

Table 11 presents the Vivado post-implementation timing summary at 200 MHz. The baseline exhibits a setup violation of -0.009 ns on one endpoint. After applying the six RTL techniques, WNS improves to $+0.070$ ns with zero failing

TABLE 10. Iso-throughput energy comparison at 9.87 FPS.

Work	Reported FPS	Reported Power [W]	Energy/frame at native FPS [J]	Energy/frame at 9.87 FPS [J]
Nguyen et al. [35]	125	26.4	0.211	2.673
Valadanzoj et al. [36]	55	13.6	0.247	1.377
Amin et al. [37]	15	3.5	0.233	0.355
Heller et al. [33]	15	8.0	0.533	0.810
This work	9.87	1.808	0.183	0.183

Energy/frame at 9.87 FPS computed as reported power divided by 9.87. Dynamic power (1.808 W) is used for this work, consistent with the Vivado Report Power methodology.

TABLE 11. Vivado design timing summary: Baseline vs. Optimized (ZCU104, 200 MHz).

Timing Metric	Baseline	Optimized	Status
WNS (worst negative slack)	−0.009 ns	+0.070 ns	Met
TNS (total negative slack)	−0.009 ns	0.000 ns	Met
Failing endpoints (setup)	1	0	Met
WHS (worst hold slack)	0.000 ns	+0.009 ns	Met
THS (total hold slack)	0.000 ns	0.000 ns	Met
Failing endpoints (hold)	0	0	Met
Total endpoints analyzed	261,318	243,778	—

endpoints across all 243,778 analyzed paths. The clock-enable comparator chains introduced by Technique 1 shift the critical path from the DSP48 output register to the register-bank change-detection logic, but the synthesized comparator tree maps efficiently onto the UltraScale+ LUT fabric without requiring additional pipeline registers or frequency relaxation.

Operation at higher frequencies such as 250 MHz or 300 MHz would require either pipeline insertion into the comparator chain or reduction of BANK_BITS to decrease comparator depth, as the current positive slack margin of +0.070 ns provides insufficient timing budget for direct frequency scaling beyond 200 MHz. This frequency scalability trade-off is identified as a direction for future work.

F. DETECTION ACCURACY

The proposed RTL optimizations suppress switching activity via clock-enable and operand-gating logic without modifying any arithmetic operation, weight value, or numerical precision. The optimized implementation is therefore guaranteed by construction to produce bit-exact outputs identical to the baseline Tensil implementation. Table 12 confirms this empirically across 595 consecutive live inference frames.

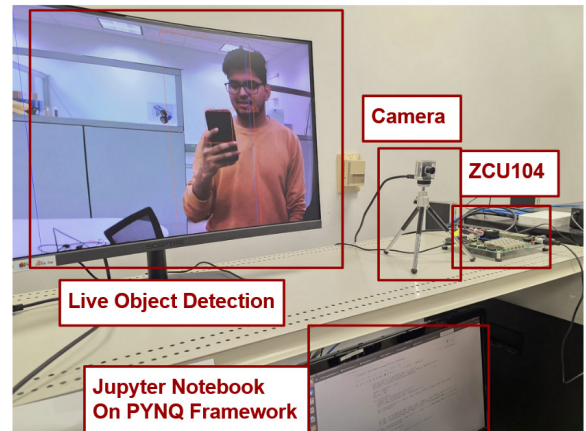
Accuracy evaluation was performed on 595 consecutive live inference frames captured via USB camera under controlled indoor lighting conditions. Ground truth annotations were established manually for four COCO classes present in the test scene: person, mobile phone, dining table, and laptop. Detection outputs were compared frame by frame between the baseline and optimized implementations using a Python script running on the PYNQ framework. A detection was considered correct if the predicted bounding box had IoU

TABLE 12. Detection accuracy evaluation: Baseline vs. Optimized.

Metric	Baseline	Optimized
Total frames evaluated	595	595
Frames with identical output	595	595
Frame-level agreement	100%	100%
4-class mAP@0.5 (indoor) [†]	99%	99%
Standard MS-COCO mAP@0.5*	21–40%	21–40%

*Weights and inference pipeline unchanged [38]; applies equally to both.

[†]Evaluated on a controlled indoor 4-class subset; not a standalone benchmark.


FIGURE 8. Real-time Tiny-YOLOv4 object detection running on ZCU104 at 9.87 FPS, showing live bounding box outputs across the four evaluated COCO classes (person, mobile phone, dining table, laptop) under controlled indoor lighting conditions.

greater than 0.5 with the ground truth annotation, consistent with the standard mAP@0.5 metric. The 99% figure reflects performance on this specific indoor 4-class subset and is not directly comparable to standard MS-COCO benchmark results.

The test scene contained four COCO classes (person, mobile phone, dining table, laptop) evaluated under controlled indoor lighting. The 99% figure reflects this specific indoor subset and is not a standalone benchmark; the standard MS-COCO reference for Tiny-YOLOv4 is 21–40% mAP@0.5 [26], applicable equally to both implementations. Note that accuracy figures reported across works in Table 13 are not directly comparable, as each work uses a different dataset and evaluation protocol. The standard MS-COCO mAP@0.5 range of 21–40% applies equally to both the baseline and optimized implementations of this work.

TABLE 13. Comparison of FPGA implementations for YOLO models.

Author	Year	Model	FPGA	Image	Acc.(%)	FPS	Power (W)	Efficiency (FPS/W)
Wang et al. [32]	2023	CNN	Spartan-6	32×32	96	16	0.79	20.25
Heller et al. [33]	2022	YOLO V4 Tiny	Kria KV260	HD	75	15	8.0	1.87
Montgomerie-Corcoran et al. [34]	2023	YOLO V3 Tiny	VCU110	416×416	—	69	15.4	4.5
Nguyen et al. [35]	2024	YOLO V4 Tiny	ZCU104	HD	78	125	26.4	4.7
Valadanzoj et al. [36]	2024	YOLO V4 Tiny	ZC706	416×416	79	55	13.6	4.0
Amin et al. [37]	2024	YOLO V3 Tiny	Kria KV260	HD	99	15	3.5	4.2
This Work	2025	YOLO V4 Tiny	ZCU104	HD	99^a	9.87	2.496	3.95 ^b

^aCOCO-trained weights identical to the Tensil baseline [22]. The 99% figure reflects live indoor verification on a 4-class subset; standard MS-COCO mAP@0.5 is 21–40% [38], applicable to both implementations equally. Accuracy figures across works are not directly comparable as each uses a different dataset and evaluation protocol.

^bFPS/W reflects ZCU104's high static power fraction at 9.87 FPS. Table 10 provides an iso-throughput comparison showing this work achieves the lowest energy per inference among all compared implementations.

G. COMPARATIVE EVALUATION

Table 13 benchmarks the proposed accelerator against recent FPGA-based YOLO implementations. The proposed work achieves the lowest reported total on-chip power (2.496 W), representing a 5.8× reduction relative to the only other ZCU104 baseline (Nguyen et al., 26.4 W [35]). The FPS/W figure (3.95) reflects the ZCU104's high static power fraction at 9.87 FPS; Table 10 confirms that this work achieves the lowest energy per inference at an equivalent throughput.

It should be noted that accuracy figures reported across works in Table 13 are not directly comparable, as each work uses a different dataset, test split, and evaluation protocol. Wang et al. report 96% on a custom 32×32 dataset; Heller et al. report 75% on a marine objects subset; Nguyen et al. and Valadanzoj et al. report results on undisclosed test sets. The 99% figure reported for both Amin et al. and this work reflects controlled indoor subsets rather than the standard MS-COCO benchmark. The standard MS-COCO mAP@0.5 range of 21–40% for Tiny-YOLOv4 [38] applies equally to both the baseline and optimized implementations of this work, confirming that the proposed RTL optimizations introduce no accuracy degradation.

H. DISCUSSION

The proposed low-power MAC architecture substantially reduces switching activity throughout the datapath. Operand-based gating, banked registers with change-detection enables, and valid-gated pipelines collectively suppress toggling in Signals, Logic, and DSP units, accounting for the majority of the power savings. The LUT increase (25.00% → 57.37%) represents the necessary overhead of fine-grained clock-enable and operand-gating control logic, consistent with the power–area trade-off inherent in RTL-level optimization. The dual-method validation (Vivado simulation and INA226 board measurement) agrees to within one percentage point, confirming that the reported reductions reflect physical switching suppression rather than simulation artifacts.

The power reductions reported in this work are unlikely to be fully replicated by modern High-Level Synthesis (HLS)

flows or Vivado's built-in power optimization alone, at least without additional manual RTL intervention. First, HLS tools operate at the algorithmic level and reduce operation count through loop unrolling, pipelining, and memory partitioning, but do not suppress idle switching within active arithmetic units. The six techniques proposed here specifically target signal transitions that occur even when valid computations are being performed, which HLS has no mechanism to detect or suppress. Second, Vivado's built-in power optimization applies global clock gating and register retiming at the netlist level, but does not have visibility into operand-level activity patterns such as repeated weight values in convolutional reuse patterns or zero-valued activations in sparse layers. The change-detect enables (`ce_bank`) and operand bypass (`skip_mul`) exploit these CNN-specific data patterns explicitly, which requires manual RTL instrumentation. Third, the accumulator upper-bit freezing technique (`ce_hi`) requires knowledge of carry propagation behaviour specific to the MAC datapath structure, which neither HLS nor Vivado power optimization can infer automatically from a high-level description.

The proposed techniques are particularly effective on DSP48-based designs because the DSP48 input register clock enables (`dsp_ce`) provide a direct suppression handle that LUT-based logic does not expose. For ultra-low-power regimes where throughput is modest, LUT-based MAC implementations could reduce static power by avoiding DSP48 leakage; a quantitative comparison is identified as a direction for future work.

The principal limitation is LUT overhead: at 57.37%, the design exceeds the capacity of smaller devices such as the Kria KV260 (117,120 LUTs), which the current 16-bit implementation exceeds by only 2.68% (120,257 LUTs required). As analysed in Section III, reducing `DATA_WIDTH` to 8 bits is estimated to reduce control-logic LUT count by approximately 40%, bringing the design comfortably within the KV260 budget. This HLS-vs-RTL comparison and the 8-bit KV260 variant are the primary directions for future work.

VI. CONCLUSION

This paper presented a MAC-centric RTL power-optimization methodology integrating six orthogonal techniques — register banking with change-detect enables, activity-based primary gating, operand-based computation bypass, adaptive frequency scaling, pipeline valid-gating, and grouped bank updates with upper-bit freezing — applied simultaneously across the full MAC datapath of a Tiny-YOLOv4 accelerator on the Xilinx ZCU104. Unlike prior work, which applies at most two such mechanisms in isolation, the proposed methodology targets all six orthogonal switching sources concurrently, confirmed by a near-additive ablation study (combined: 59.17%; sum of individuals: 59.15%).

The methodology achieves a 59.2% reduction in dynamic power and a 51.4% reduction in total on-chip power, with energy per frame improving by 59.2% at 9.87 FPS. Detection accuracy is preserved by construction — no weights, arithmetic operations, or numerical precision are modified — confirmed bit-exact across 595 live inference frames. On-board INA226 validation confirms a 62.5% reduction in inference-induced PL switching power, consistent with Vivado simulation estimates to within one percentage point. At the iso-throughput operating point of 9.87 FPS, the proposed work achieves the lowest energy per inference (0.183 J) among all compared FPGA-based YOLO implementations. The primary resource cost is increased LUT utilization (25% → 57%), while BRAM, DSP48, and BUFG resources remain completely unchanged.

Future work will focus on five directions: (1) porting the techniques to the Kria KV260 at 8-bit precision to validate power efficiency under tighter LUT constraints; (2) extending the methodology to YOLOv7, YOLOv8, and YOLOv9 with standard MS-COCO accuracy validation; (3) generating SAIF-based switching activity profiles across diverse workloads with variable sparsity to quantify sensitivity to real-world activation distributions; (4) quantitative comparison of the proposed techniques applied to LUT-based MAC implementations in ultra-low-power regimes; and (5) evaluating the gap between the proposed manual RTL approach and HLS-generated baselines augmented with vendor power optimization pragmas.

ACKNOWLEDGMENT

The authors thank their colleagues from HL Mando and KEIT, who provided insight and expertise that greatly assisted the research and improved the manuscript.

REFERENCES

- [1] A. K. Jameil and H. Al-Rawashdy, "Efficient CNN architecture on FPGA using high level module for healthcare devices," *IEEE Access*, vol. 10, pp. 60486–60495, 2022, doi: [10.1109/ACCESS.2022.3180829](https://doi.org/10.1109/ACCESS.2022.3180829).
- [2] S. Y. Nikouei, Y. Chen, S. Song, R. Xu, B.-Y. Choi, and T. Faughnan, "Smart surveillance as an edge network service: From harr-cascade, SVM to a lightweight CNN," in *Proc. IEEE 4th Int. Conf. Collaboration Internet Comput. (CIC)*, Oct. 2018, pp. 256–265.
- [3] K. Haeublein, W. Brueckner, S. Vaas, S. Rachuj, M. Reichenbach, and D. Fey, "Utilizing PYNQ for accelerating image processing functions in ADAS applications," in *Proc. ARCS Workshop; 32nd Int. Conf. Archit. Comput. Syst.*, Copenhagen, Denmark, May 2019, pp. 1–8.
- [4] Z. Zhang, M. A. P. Mahmud, and A. Z. Kouzani, "FitNN: A low-resource FPGA-based CNN accelerator for drones," *IEEE Internet Things J.*, vol. 9, no. 21, pp. 21357–21369, Nov. 2022, doi: [10.1109/JIOT.2022.3179016](https://doi.org/10.1109/JIOT.2022.3179016).
- [5] C. Fu and Y. Yu, "FPGA-based power efficient face detection for mobile robots," in *Proc. IEEE Int. Conf. Robot. Biomimetics (ROBIO)*, Dali, Dali, China, Dec. 2019, pp. 467–473, doi: [10.1109/ROBIO49542.2019.8961745](https://doi.org/10.1109/ROBIO49542.2019.8961745).
- [6] X. Li, X. Gong, D. Wang, J. Zhang, T. Baker, J. Zhou, and T. Lu, "ABM-SpConv-SIMD: Accelerating convolutional neural network inference for industrial IoT applications on edge devices," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 4, pp. 2736–2749, Jul. 2022, doi: [10.1109/TNSE.2022.3154412](https://doi.org/10.1109/TNSE.2022.3154412).
- [7] S. Tamimi, Z. Ebrahimi, B. Khaleghi, and H. Asadi, "An efficient SRAM-based reconfigurable architecture for embedded processors," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 3, pp. 466–479, Mar. 2019.
- [8] A. J. A. El-Maksoud, M. Ebbad, A. H. Khalil, and H. Mostafa, "Power efficient design of high-performance convolutional neural networks hardware accelerator on FPGA: A case study with GoogLeNet," *IEEE Access*, vol. 9, pp. 151897–151911, 2021, doi: [10.1109/ACCESS.2021.3126832](https://doi.org/10.1109/ACCESS.2021.3126832).
- [9] S. Lee, D. Kim, D. Nguyen, and J. Lee, "Double MAC on a DSP: Boosting the performance of convolutional neural networks on FPGAs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 5, pp. 888–897, May 2019, doi: [10.1109/TCAD.2018.2824280](https://doi.org/10.1109/TCAD.2018.2824280).
- [10] S. Ullah, S. Rehman, M. Shafique, and A. Kumar, "High-performance accurate and approximate multipliers for FPGA-based hardware accelerators," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 2, pp. 211–224, Feb. 2022, doi: [10.1109/TCAD.2021.3056337](https://doi.org/10.1109/TCAD.2021.3056337).
- [11] X. Wu, Y. Ma, M. Wang, and Z. Wang, "A flexible and efficient FPGA accelerator for various large-scale and lightweight CNNs," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 3, pp. 1185–1198, Mar. 2022, doi: [10.1109/TCSI.2021.3131581](https://doi.org/10.1109/TCSI.2021.3131581).
- [12] W. Liu, J. Lin, and Z. Wang, "A precision-scalable energy-efficient convolutional neural network accelerator," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 10, pp. 3484–3497, Oct. 2020, doi: [10.1109/TCSI.2020.2993051](https://doi.org/10.1109/TCSI.2020.2993051).
- [13] H. Irmak, D. Ziener, and N. Alachiotis, "Increasing flexibility of FPGA-based CNN accelerators with dynamic partial reconfiguration," in *Proc. 31st Int. Conf. Field-Program. Log. Appl. (FPL)*, Dresden, Germany, Aug. 2021, pp. 306–311, doi: [10.1109/FPL53798.2021.00061](https://doi.org/10.1109/FPL53798.2021.00061).
- [14] W. Chen, D. Wang, H. Chen, S. Wei, A. He, and Z. Wang, "An asynchronous and reconfigurable CNN accelerator," in *Proc. IEEE Int. Conf. Electron Devices Solid State Circuits (EDSSC)*, Shenzhen, China, Jun. 2018, pp. 1–2, doi: [10.1109/EDSSC.2018.8487115](https://doi.org/10.1109/EDSSC.2018.8487115).
- [15] C. Yang, Y. Wang, H. Zhang, X. Wang, and L. Geng, "A reconfigurable CNN accelerator using tile-by-tile computing and dynamic adaptive data truncation," in *Proc. IEEE Int. Conf. Integr. Circuits, Technol. Appl. (ICTA)*, Chengdu, China, Nov. 2019, pp. 73–74.
- [16] S. Zeng, K. Guo, S. Fang, J. Kang, D. Xie, Y. Shan, Y. Wang, and H. Yang, "An efficient reconfigurable framework for general purpose CNN-RNN models on FPGAs," in *Proc. IEEE 23rd Int. Conf. Digit. Signal Process. (DSP)*, Nov. 2018, pp. 1–5.
- [17] L. Gong, C. Wang, X. Li, H. Chen, and X. Zhou, "MALOC: A fully pipelined FPGA accelerator for convolutional neural networks with all layers mapped on chip," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 11, pp. 2601–2612, Nov. 2018, doi: [10.1109/TCAD.2018.2857078](https://doi.org/10.1109/TCAD.2018.2857078).
- [18] L. Bai, Y. Zhao, and X. Huang, "A CNN accelerator on FPGA using depthwise separable convolution," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 65, no. 10, pp. 1415–1419, Oct. 2018, doi: [10.1109/TCSII.2018.2865896](https://doi.org/10.1109/TCSII.2018.2865896).
- [19] J. Qiu, J. Wang, S. Yao, K. Guo, B. Li, E. Zhou, J. Yu, T. Tang, N. Xu, S. Song, Y. Wang, and H. Yang, "Going deeper with embedded FPGA platform for convolutional neural network," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2016, pp. 26–35, doi: [10.1145/2847263.2847265](https://doi.org/10.1145/2847263.2847265).
- [20] Y. Ma, Y. Cao, S. Vruthula, and J.-S. Seo, "Optimizing the convolution operation to accelerate deep neural networks on FPGA," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 26, no. 7, pp. 1354–1367, Jul. 2018, doi: [10.1109/TVLSI.2018.2815603](https://doi.org/10.1109/TVLSI.2018.2815603).
- [21] S. Li, Y. Luo, K. Sun, N. Yadav, and K. K. Choi, "A novel FPGA accelerator design for real-time and ultra-low power deep convolutional neural networks compared with Titan X GPU," *IEEE Access*, vol. 8, pp. 105455–105471, 2020, doi: [10.1109/ACCESS.2020.3000009](https://doi.org/10.1109/ACCESS.2020.3000009).

- [22] Tensil AI. *Tensil Tutorial for YOLO V4 Tiny on Ultra96 V2*. Accessed: Dec. 15, 2022. [Online]. Available: <https://k155la3.blog/2022/04/04/tensil-tutorial-for-yolo-v4-tiny-on-ultra96-v2/>
- [23] X. Zhang, Y. Ma, J. Xiong, W. W. Hwu, V. Kindratenko, and D. Chen, "Exploring HW/SW co-design for video analysis on CPU-FPGA heterogeneous systems," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 6, pp. 1606–1619, Jun. 2022, doi: [10.1109/TCAD.2021.3093398](https://doi.org/10.1109/TCAD.2021.3093398).
- [24] G. Lakshminarayanan and B. Venkataramani, "Optimization techniques for FPGA-based wave-pipelined DSP blocks," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 13, no. 7, pp. 783–792, Jul. 2005, doi: [10.1109/TVLSI.2005.850086](https://doi.org/10.1109/TVLSI.2005.850086).
- [25] D. Wang, K. Xu, J. Guo, and S. Ghiasi, "DSP-efficient hardware acceleration of convolutional neural network inference on FPGAs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 12, pp. 4867–4880, Dec. 2020, doi: [10.1109/TCAD.2020.2968023](https://doi.org/10.1109/TCAD.2020.2968023).
- [26] A. Prihozhy, E. Bezati, A. A. Ab Rahman, and M. Mattavelli, "Synthesis and optimization of pipelines for HW implementations of dataflow programs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 34, no. 10, pp. 1613–1626, Oct. 2015, doi: [10.1109/TCAD.2015.2427278](https://doi.org/10.1109/TCAD.2015.2427278).
- [27] W. Lou, L. Gong, C. Wang, Z. Du, and X. Zhou, "OctCNN: A high throughput FPGA accelerator for CNNs using octave convolution algorithm," *IEEE Trans. Comput.*, vol. 71, no. 8, pp. 1847–1859, Aug. 2022, doi: [10.1109/TC.2021.3110413](https://doi.org/10.1109/TC.2021.3110413).
- [28] H. Kim and K. Choi, "Low power FPGA-SoC design techniques for CNN-based object detection accelerator," in *Proc. IEEE 10th Annu. Ubiquitous Comput., Electron. Mobile Commun. Conf. (UEMCON)*, New York, NY, USA, Oct. 2019, pp. 1130–1134, doi: [10.1109/UEMCON47517.2019.8992929](https://doi.org/10.1109/UEMCON47517.2019.8992929).
- [29] Y. Kim, H. Kim, N. Yadav, S. Li, and K. K. Choi, "Low-power RTL code generation for advanced CNN algorithms toward object detection in autonomous vehicles," *Electronics*, vol. 9, no. 3, p. 478, Mar. 2020, doi: [10.3390/electronics9030478](https://doi.org/10.3390/electronics9030478).
- [30] H. Kim and K. Choi, "The implementation of a power efficient BCNN-based object detection acceleration on a Xilinx FPGA-SoC," in *Proc. Int. Conf. Internet Things (iThings) IEEE Green Comput. Commun. (GreenCom) IEEE Cyber, Phys. Social Comput. (CPSCOM) IEEE Smart Data (SmartData)*, Atlanta, GA, USA, Jul. 2019, pp. 240–243, doi: [10.1109/ITHINGS/GREENCOM/CPSCOM/SMARTDATA.2019.00060](https://doi.org/10.1109/ITHINGS/GREENCOM/CPSCOM/SMARTDATA.2019.00060).
- [31] Y. Kim, Q. Tong, K. Choi, E. Lee, S.-J. Jang, and B.-H. Choi, "System level power reduction for YOLO2 sub-modules for object detection of future autonomous vehicles," in *Proc. Int. SoC Design Conf. (ISOC)*, Nov. 2018, pp. 151–155, doi: [10.1109/ISOC.2018.8649950](https://doi.org/10.1109/ISOC.2018.8649950).
- [32] Y. Wang, Y. Liao, J. Yang, H. Wang, Y. Zhao, C. Zhang, B. Xiao, F. Xu, Y. Gao, M. Xu, and J. Zheng, "An FPGA-based online reconfigurable CNN edge computing device for object detection," *Microelectron. J.*, vol. 137, Jul. 2023, Art. no. 105805, doi: [10.1016/j.mejo.2023.105805](https://doi.org/10.1016/j.mejo.2023.105805).
- [33] D. Heller, M. Rizk, R. Douguet, A. Baghdadi, and J.-P. Diguët, "Marine objects detection using deep learning on embedded edge devices," in *Proc. IEEE Int. Workshop Rapid Syst. Prototyping (RSP)*, Shanghai, China, Oct. 2022, pp. 1–7, doi: [10.1109/RSP57251.2022.10039025](https://doi.org/10.1109/RSP57251.2022.10039025).
- [34] A. Montgomerie-Corcoran, P. Toupas, Z. Yu, and C.-S. Bouganis, "SATAY: A streaming architecture toolflow for accelerating YOLO models on FPGA devices," in *Proc. Int. Conf. Field Program. Technol. (ICFPT)*, Yokohama, Japan, Dec. 2023, pp. 179–187, doi: [10.1109/icfpt59805.2023.00025](https://doi.org/10.1109/icfpt59805.2023.00025).
- [35] D.-D. Nguyen, D.-T. Nguyen, M.-T. Le, and Q.-C. Nguyen, "FPGA-SoC implementation of YOLOv4 for flying-object detection," *J. Real-Time Image Process.*, vol. 21, no. 3, Jun. 2024, doi: [10.1007/s11554-024-01440-w](https://doi.org/10.1007/s11554-024-01440-w).
- [36] Z. Valadanjoz, H. Daryanavard, and A. Harifi, "High-speed YOLOv4-tiny hardware accelerator for self-driving automotive," *J. Supercomput.*, vol. 80, no. 5, pp. 6699–6724, Mar. 2024, doi: [10.1007/s11227-023-05713-2](https://doi.org/10.1007/s11227-023-05713-2).
- [37] R. A. Amin, M. Hasan, V. Wiese, and R. Obermaier, "FPGA-based real-time object detection and classification system using YOLO for edge computing," *IEEE Access*, vol. 12, pp. 73268–73278, 2024, doi: [10.1109/ACCESS.2024.3404623](https://doi.org/10.1109/ACCESS.2024.3404623).
- [38] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," 2020, *arXiv:2004.10934*.
- [39] A. Rodríguez, C. Palmero, J. Cortadella, and J. Carrabina, "Efficient Edge AI: Deploying convolutional neural networks on FPGA with the gemmini accelerator," 2024, *arXiv:2408.07404*.

- [40] H. Wang, D. Li, and T. Isshiki, "Energy-efficient implementation of YOLOv8, instance segmentation, and pose detection on RISC-V SoC," *IEEE Access*, vol. 12, pp. 64050–64068, 2024, doi: [10.1109/ACCESS.2024.3397536](https://doi.org/10.1109/ACCESS.2024.3397536).



computing-in-memory architectures, VLSI systems, and hardware-aware machine learning.

ACHYUTH GUNDRAPALLY (Graduate Student Member, IEEE) received the B.Tech. degree in electronics and communication engineering from Indian Institute of Information Technology, Sri City, Chittoor, India, in 2022, and the M.S. degree in electrical engineering from Illinois Institute of Technology, Chicago, IL, USA, in 2025, where he is currently pursuing the Ph.D. degree in electrical engineering. His current research interests include FPGA acceleration, low-power hardware design,



system levels, as well as hardware and software co-design for low-power and high-speed machine learning accelerators on FPGA platforms.

YATRIK ASHISH SHAH (Graduate Student Member, IEEE) received the B.S. degree in instrumentation and control engineering from the Government Engineering College, Gandhinagar, India, in 2022, and the M.S. degree in computer engineering from Illinois Institute of Technology, Chicago, USA. He is currently a Research Associate with the VLSI Design and Automation Laboratory. His current research interests include high-performance and ultra-low-power design at the register-transfer and



laboratory, Institute of Industrial Science, The University of Tokyo, Japan, working on leakage-power-reduction circuit techniques. He has proposed and conducted several projects supported by the National Aeronautics and Space Administration (NASA), the Defense Advanced Research Projects Agency (DARPA), U.S. National Science Foundation (NSF), Scientific Research Corporation (SRC), and Korea Electronics Technology Institute (KETI) regarding Power-Aware Computing/Communication (PACC). He was a Senior CAD Engineer and a Technical Consultant for low-power systems-on-chip (SoC) design with Samsung Semiconductor, Broadcom, and Sequence Design, before joining IIT. In the past, he had eight years of industry experience in ultra-low-power VLSI chip design from the compiler level to the circuit level. In the last few years, using his novel low-power techniques, several processors and mobile chips have been successfully fabricated in deep-submicrometer technology, and more than 120 peer-reviewed journals and conference papers have been published. His current research interests include ultra-low-power digital circuits and artificial intelligence (AI) related IC designs. He is the Editor-in-Chief of *Journal of Pervasive Technologies*; a guest editor of Springer and Wiley journals; and a TPC member for several IEEE circuit design conferences.

...